0062. Unique Paths

n | Category: 2-D Dynamic Programming | LeetCode: https://leetcode.com/proble

Problem

- A robot moves only right or down on an m x n grid from the top-left to the bottom-right. Count the paths.

Intuition

- Paths(r, c) = paths(r-1, c) + paths(r, c-1). The first row and column have exactly 1 path.
- One row of the table is enough. (Closed form: C(m+n-2, m-1).)

Brute Force

- Recursion without memo: O(2^(m+n)).

Optimal Approach

- Row = [1] n.
- For each of the remaining m-1 rows: for c = 1..n-1: row[c] += row[c-1].

Worked Example

```
m = 3, n = 7.

row0: 1 1 1 1  1  1  1
row1: 1 2 3 4  5  6  7
row2: 1 3 6 10 15 21 28   -> 28
```

Java Solution

```java
public class UniquePaths {
    public int uniquePaths(int m, int n) {
        int[] row = new int[n];
        Arrays.fill(row, 1);
        for (int r = 1; r < m; r++)
            for (int c = 1; c < n; c++) row[c] += row[c - 1];
        return row[n - 1];
    }
}
```

Complexity

- Time: O(m n). Space: O(n).

Pitfalls

- The combinatorial formula overflows int quickly; use long and incremental division if you go that way.